

Cognitive Memory for AI Agents: Decay Floors, Emergent Core Memories, and Retrieval-Driven Reinforcement

Bhekani Khumalo
Independent Researcher
hello@bhekani.com
<https://www.bhekani.com>

May 2026

Abstract

Most AI memory systems focus on two lifecycle edges: deciding what to extract and deciding how to retrieve it. The middle—what happens after a memory is stored but before it is retrieved—remains less settled. This paper presents **cognitive-memory**, an open-source memory architecture for LLM-based agents that treats memory as a dynamic lifecycle rather than a static vector index. The system combines decay floors, emergent core-memory promotion, two-tier retrieval reinforcement, associative linking, tiered hot/cold/stub storage, and reversible consolidation. Faint memories can become hard to retrieve without being immediately discarded, and retrieval itself changes future accessibility. We situate the design in cognitive-science work on forgetting, availability versus accessibility, and spreading activation, then evaluate it on LoCoMo, LongMemEval-S, and a controlled long-term-interaction benchmark. On LoCoMo, empirically tuned **cognitive-memory** defaults reach 46.2% overall F1 and 51.3% multi-hop F1; on LongMemEval-S, it reaches 71.6% task-averaged accuracy, competitive with ENGRAM-class single-stage systems but behind higher-scoring or differently configured recent architectures. A controlled benchmark retains 100% of identity-critical facts across a 30-day mixed-access window, while also exposing likely over-promotion to core status.

1 Introduction

Large language models are stateless. Every inference is a fresh start. To build agents that maintain continuity across sessions, developers must bolt on external memory systems—and a growing ecosystem of tools exists to do exactly that [Chhikara et al., 2025, Packer et al., 2023, Sumers et al., 2024]. These systems have gotten quite good at two things: extracting salient information from conversations, and retrieving relevant memories when prompted. But between extraction and retrieval lies a dead zone. Once stored, memories sit in a database with the same weight and accessibility they had on the day they were created.

This is not how biological memory works. Human memory is dynamic: it strengthens with use, weakens with neglect, and selectively preserves information that has proven important over time [Ebbinghaus, 1885, Bjork and Bjork, 1992]. Crucially, forgetting is not a failure mode but an adaptive feature. Richards and Frankland [2017] argue that the goal of memory is not faithful information transmission but rather guiding intelligent decision-making in noisy, changing environments. Partial forgetting enables generalization by stripping away context-specific details and preserving underlying patterns.

For AI agents, the engineering consequences of ignoring this middle phase are concrete. A memory system that retains everything at full strength produces context windows bloated with

noise, stale facts weighted equally to fresh ones, and no way to distinguish a recurring preference from a passing remark. Consolidation—compressing old memories into summaries—is the field’s most common response, but it is lossy and irreversible: the original memories are destroyed, and with them the ability to recover details that may become relevant again.

This paper presents an alternative approach. Our system, **cognitive-memory**, implements biologically-inspired decay that dims memories without immediately destroying them, retrieval-driven reinforcement that strengthens memories through use, and emergent core memory detection that protects critical information. The key philosophical commitment is that **a memory should not be pruned merely because it has become hard to retrieve**. Instead, memories fade to a floor: regular memories to a faint but non-zero level, core memories to a high floor that keeps them always accessible. This preserves the possibility of reactivation during the memory’s active or cold-storage window—the everyday experience of a forgotten detail flooding back when triggered by the right cue.

We contribute: (1) a formal architecture grounded in specific cognitive science mechanisms; (2) a particular combination of lifecycle design elements—decay floors, emergent core memory promotion, reversible consolidation, and two-tier retrieval boosting—that is not commonly treated as a first-class memory-management layer; (3) an open-source implementation available as TypeScript and Python SDKs; (4) a description of its deployment in a production chat application, `blah.chat`,¹ and (5) an empirical evaluation on LoCoMo, LongMemEval-S, and a controlled long-term-interaction benchmark targeted at the architectural claims (Section 6).

2 Related Work

2.1 Taxonomic Foundations

The CoALA framework [Sumers et al., 2024] provides the most influential taxonomy for agent memory, distinguishing working memory (short-term, in-context) from long-term memory (episodic, semantic, procedural). CoALA draws on the SOAR cognitive architecture and identifies memory management as a key area for future development, though its focus is primarily on the structural organization of memory rather than temporal dynamics.

2.2 Memory Architectures for Agents

Park et al. [2023] introduced the generative agents architecture, where agents maintain a “memory stream” of observations and synthesize higher-level reflections over time. Their retrieval function combines recency, importance, and relevance—an early recognition that not all memories should be weighted equally. However, the recency signal is a simple time-based decay with no mechanism for retrieval-driven strengthening or differentiated decay rates.

MemGPT [Packer et al., 2023] models agent memory on an operating system’s virtual memory hierarchy, with a core memory (in-context) and archival memory (out-of-context) layer. The agent manages its own context through explicit read/write operations. This is an architectural innovation in *how* memory is organized, but it does not address the temporal dynamics of what happens to memories over time.

Mem0 [Chhikara et al., 2025] provides a unified memory layer with extraction, deduplication, and retrieval. Recent versions describe decay mechanisms where “memories strengthen when recalled and naturally decay over time if unused,” though the implementation details and decay model are not fully specified in published materials.

¹<https://blah.chat>

2.3 Forgetting Mechanisms

The most directly related work is FadeMem [Wei et al., 2026], published in January 2026. FadeMem implements Ebbinghaus-inspired exponential decay across a dual-layer (short-term/long-term) memory hierarchy, with retention modulated by semantic relevance, access frequency, and temporal patterns. It incorporates LLM-guided conflict resolution and memory fusion, and demonstrates improvements over Mem0 and MemGPT on multiple benchmarks. Our work shares the same cognitive science inspiration but differs in several design choices detailed in Section 4.

MemoryBank [Zhong et al., 2024] incorporates the Ebbinghaus forgetting curve to model memory strength decay and supports continual memory updating and user profile construction. SAGE [Chen et al., 2024] combines the Ebbinghaus curve with linguistic knowledge to optimize memory management, applying part-of-speech priority to determine information retention. Memoria [Lee and Jang, 2023] addresses what it calls the “fateful forgetting problem” through a human-inspired memory architecture. MemOS [Kang et al., 2025] introduces TTL-based lifespan policies and state transitions (Generated → Activated → Merged → Archived → Expired).

Recent and adjacent systems. Several systems published shortly before or during our evaluation window sit close to our problem framing on long-horizon conversational memory benchmarks. ENGRAM [Patel and Patel, 2025] routes turns into episodic, semantic, and procedural records and reports strong LongMemEval-S accuracy at the time it was posted; it is the closest single-stage baseline against which we compare. TiMem [Li et al., 2026] organises conversations through a Temporal Memory Tree that consolidates raw observations into progressively abstracted persona representations and reports 76.88% on LongMemEval-S, exceeding ENGRAM and our own 71.6%. EverMemOS [Hu et al., 2026] implements an engram-inspired lifecycle (episodic trace formation, semantic consolidation, reconstructive recollection) and reports 83.0% on LongMemEval. MAGMA [Jiang et al., 2026] represents each memory across semantic, temporal, causal, and entity graphs, retrieving by query-adaptive graph traversal rather than by a single flat similarity search. Hindsight [Latimer et al., 2025] treats memory as a structured substrate for reasoning, separating facts, experiences, entity summaries, and evolving beliefs, and reports substantially higher LongMemEval and LoCoMo results under larger or different backbone settings than those used here.

The closest adjacent work to our preservation and scaling claims is All-Mem [Lv et al., 2026]. All-Mem maintains a topology-structured memory bank with non-destructive consolidation, a bounded visible surface for online retrieval, and hop-bounded expansion from active anchors to archived evidence. This overlaps with our reversible consolidation and hot/cold retrieval motivation, but differs in mechanism: All-Mem evolves memory topology through offline SPLIT, MERGE, and UPDATE operations, whereas **cognitive-memory** focuses on continuous retention dynamics, decay floors, retrieval-driven reinforcement, emergent core promotion, and explicit availability-accessibility scoring.

TiMem, EverMemOS, MAGMA, Hindsight, and All-Mem all pre-date publication of this paper, though several post-date our evaluation runs. We acknowledge them here so that our LongMemEval-S result is not over-claimed: it is competitive with ENGRAM-class single-stage systems, but it is not a leaderboard result as of mid-2026. Our contribution is an architectural one—decay floors, emergent core promotion, two-tier boosting, and graceful hot/cold/stub degradation—rather than a benchmark-leading numerical one, and the empirical results in Section 6 are reported as evidence that the architecture behaves as designed at competitive performance levels.

The “Memory in the Age of AI Agents” survey [Liu et al., 2025] provides the most comprehensive recent overview, categorizing forgetting as one of three branches of memory evolution alongside consolidation and updating. That survey’s taxonomy distinguishes memory by *form*

(token-level, parametric, latent) and *function* (factual, experiential, working). Our system operates entirely at the token level—memories are discrete text records in an external store—and primarily serves the factual memory function, maintaining persistent knowledge about users and their preferences across sessions. This confirms that forgetting is an active area of research, though one still in its early stages with no dominant approach.

2.4 Cognitive Science Foundations

Our architecture draws on several established results from cognitive science and neuroscience:

- **Ebbinghaus forgetting curves** [Ebbinghaus, 1885, Murre and Dros, 2015]: Memory retention is often modelled by smooth decreasing curves, with the steepest drop occurring shortly after encoding. Exponential decay is a useful engineering prior; power-law curves may better capture longer tails in some settings.
- **Spaced repetition effects** [Cepeda et al., 2006]: Memories accessed at increasing intervals develop greater long-term stability than those accessed in massed practice.
- **Spreading activation** [Collins and Loftus, 1975]: Retrieving one memory activates semantically related memories, with activation strength decreasing with associative distance.
- **Synaptic tagging** [Frey and Morris, 1997]: Certain memories receive biochemical tags during encoding that mark them for preferential long-term consolidation, independent of subsequent access patterns.
- **Adaptive forgetting** [Richards and Frankland, 2017]: The interaction between persistence and transience optimizes memory-guided decision-making. The goal of memory is not accurate recording but flexible behaviour in changing environments.
- **Intentional forgetting** [Popov et al., 2019]: Forgetting can be an effortful, resource-consuming process that actively aids memory for target items. Our system implements only passive decay rather than directed forgetting, but this work motivates the broader view that forgetting is functional rather than merely a failure of retention.
- **Availability vs. accessibility** [Tulving and Pearlstone, 1966]: Apparent forgetting often reflects a failure of retrieval (accessibility) rather than a loss of stored information (availability). Memories may be intact in storage but inaccessible without appropriate cues.
- **Complementary Learning Systems** [McClelland et al., 1995]: The brain maintains two complementary memory stores—a fast-learning, fast-decaying hippocampal system and a slow-learning, stable neocortical system—with gradual migration of memories between them.
- **Silent engrams** [Ryan and Frankland, 2022]: Forgotten memories may persist as engram ensembles that are available but inaccessible through natural retrieval cues, recoverable only through artificial stimulation. This suggests that much natural forgetting is a state change in accessibility rather than erasure.

3 Architecture

The **cognitive-memory** system operates as a middleware layer between an LLM-based agent and its vector store. When the agent produces or encounters information worth remembering, the system extracts structured memory objects, assigns initial parameters, stores them with

embeddings, and manages their lifecycle over time. On retrieval, the system queries semantically, applies retention-weighted scoring, boosts accessed memories, and returns results ordered by composite relevance.

3.1 Memory Representation

Each memory is represented as a structured object with the following fields:

Field	Type	Description
<code>content</code>	string	Natural language content
<code>category</code>	enum	episodic, semantic, procedural, core
<code>importance</code>	float [0,1]	LLM-assessed salience at extraction
<code>stability</code>	float [0,1]	Resistance to decay; increases with use
<code>accessCount</code>	int	Number of times retrieved
<code>lastAccessedAt</code>	datetime	Timestamp of most recent retrieval
<code>createdAt</code>	datetime	Timestamp of creation
<code>associations</code>	list	Links to related memories with weights

Table 1: Memory object schema. The **core** category triggers elevated decay floors and is assigned either by LLM classification at extraction or by emergent promotion (Section 3.4).

3.2 Decay Model

Retention is computed as a configurable forgetting curve with a non-zero floor. The default SDK configuration uses a modified exponential decay function:

$$R(m) = \max \left(f_m, \exp \left(\frac{-\Delta t(m)}{S(m) \cdot B(m) \cdot \tau_c} \right) \right) \quad (1)$$

where:

- $R(m) \in [f_m, 1]$ is the retention score for memory m
- f_m is the *decay floor* (see Section 3.3)
- $\Delta t(m)$ is days since last access
- $S(m) \in [0, 1]$ is the stability parameter, representing resistance to decay. In implementation, the denominator uses $S_{\text{eff}} = \max(S(m), \epsilon)$ with small $\epsilon > 0$ to avoid division by zero for newly-created or malformed records.
- $B(m) = 1 + (\text{importance}(m) \times 2)$ is the importance boost. If importance has already been normalized to $[0, 1]$, this lies in $[1, 3]$; the implementation also clamps it to 3 as a guardrail for adapters that ingest unnormalized importance scores.
- τ_c is the base retention time constant, dependent on memory category

We also implement a power-law variant used in the ablations in Section 6.5:

$$R_{\text{power}}(m) = \max \left(f_m, \left(1 + \frac{\Delta t(m)}{S(m) \cdot B(m) \cdot \tau_c} \right)^{-\gamma} \right) \quad (2)$$

where γ controls tail heaviness. The current benchmark comparison uses $\gamma = 1/\ln 2 \approx 1.4427$, chosen so that the power-law curve matches the exponential curve’s one-time-constant e-folding point: at $\Delta t = SB\tau_c$, retention is approximately $e^{-1} \approx 0.368$. We retain the exponential form as the default because it is simpler and has been characterised across the full headline run; the

power-law result is promising but currently measured only in preliminary single-conversation ablations.

Base retention time constants are differentiated by category to reflect the different temporal profiles of memory types documented in cognitive science [Finley, 2025]:

Category	τ_c (days)	Rationale
Episodic	30	Event memories fade relatively quickly
Semantic	90	Factual knowledge persists longer
Procedural	∞	Skills and procedures do not decay; raw retention is fixed at 1.0
Core	90	High floor dominates; base rate matters less

Table 2: Base retention time constants by memory category. Larger values produce slower forgetting.

3.3 Decay Floors: Never Zero by Score

The central design commitment of this architecture is that **no memory reaches zero by decay score alone**. The decay floor f_m enforces a minimum retention score:

$$f_m = \begin{cases} 0.60 & \text{if } \text{category}(m) = \text{core} \\ 0.02 & \text{otherwise} \end{cases} \quad (3)$$

A regular memory at its floor (2% retention) is unlikely to surface in normal retrieval, but it is not annihilated by the scoring function. Retrieval uses a softened accessibility term, R^α with default $\alpha = 0.3$, so a floor-retained memory has $0.02^{0.3} \approx 0.309$ accessibility. With a highly specific semantic match of 0.9, its final score is therefore approximately $0.9 \times 0.309 \approx 0.278$, rather than 0.018 under linear retention weighting. We term memories below 15% retention *faint*—rarely surfaced, but not pruned by decay alone.

This stands in deliberate contrast to systems that immediately prune memories below a threshold. FadeMem, for instance, imposes hard capacity caps (1000 long-term memories, 500 short-term) and allows memories to decay to removal [Wei et al., 2026]. Our position is not that full text should be retained forever in every deployment; Section 3.9 describes cold-storage TTL and stub conversion. The stronger claim is that decay should first degrade accessibility, not abruptly erase recoverability. The cost of retaining faint text records for a configurable dormancy window is usually small; the cost of permanently losing them before their future relevance is known is hard to predict.

This mirrors one important strand of the neurobiological evidence: many apparent memory failures reflect weakened access rather than complete erasure [Wixted, 2004]. The common experience of a long-forgotten detail returning vividly when triggered by the right cue—a smell, a name, a location—is evidence that at least some memories are dimmed, not deleted.

To make this concrete: consider a user who mentions their cat’s name in session 3, then never references it again. By session 20, that memory has decayed to its 2% floor. In a hard-pruning system, it may be gone. In ours, it persists as a faint memory during its active or cold-storage window, invisible to most normal retrieval. But when the user asks in session 47, “what was my cat called?”—a query with high semantic similarity to exactly that memory—the retention floor allows it to surface. The retrieval boost then begins restoring its stability, and the memory re-enters active circulation. This is the specific behaviour that decay floors are designed to preserve before archival stub conversion occurs.

3.4 Core Memory Detection

Core memories are those that should remain highly accessible regardless of access patterns: a user’s name, critical allergies, fundamental preferences. We implement two complementary pathways to core status. The first is loosely analogous to synaptic tagging [Frey and Morris, 1997], in that certain memories receive preferential treatment at encoding time, though our mechanism (LLM classification) is far simpler than the biological process. The second pathway mirrors use-dependent consolidation:

Explicit tagging. At extraction time, the LLM classifies whether a memory represents identity-critical information (names, medical conditions, family relationships). If so, it is immediately assigned `category = core` with the 60% floor.

Emergent promotion. Memories that were not initially tagged as core can earn core status through sustained use. A memory is promoted to core when it meets three criteria simultaneously:

1. **High access count:** retrieved more than a configurable threshold number of times (default: 10)
2. **High stability:** the stability parameter exceeds a threshold (default: 0.85)
3. **Cross-session persistence:** the memory has been accessed across at least 3 distinct sessions

This emergent pathway reflects the cognitive science observation that core memories in humans are not all tagged at encoding. Many become central to identity through repeated retrieval and emotional reinforcement over time [Bjork and Bjork, 1992]. A user’s preference for a particular communication style might not seem identity-critical when first expressed, but after being accessed in dozens of conversations, it has functionally become core knowledge about that user.

3.5 Retrieval Scoring

When the agent queries memory, results are scored by the product of semantic relevance and a power-law accessibility transform of retention:

$$\text{score}(m, q) = \text{sim}(m, q) \times R(m)^\alpha \quad (4)$$

where $\text{sim}(m, q)$ is cosine similarity clipped to $[0, 1]$, $R(m)$ is the retention score from Equation 1, and α controls how aggressively retention suppresses retrieval (default: $\alpha = 0.3$). This means two memories with identical semantic content can produce different retrieval scores based on their temporal profiles, while faint memories remain accessible to highly specific cues.

This multiplicative interaction is the mechanism by which the system implements what Richards and Frankland [2017] describe as memory’s role in guiding intelligent decisions: recent, repeatedly-accessed information is foregrounded, while stale or rarely-used information recedes without being eliminated.

3.6 Two-Tier Retrieval Boosting

When a memory is retrieved, its temporal parameters are updated to reflect the access event. We distinguish two tiers of retrieval, inspired by the cognitive distinction between active recall and passive priming via spreading activation [Collins and Loftus, 1975]:

Direct retrieval. When a memory is returned as a primary result for a query, it receives a full boost:

$$\text{accessCount}(m) \leftarrow \text{accessCount}(m) + 1 \quad (5)$$

$$\text{stability}(m) \leftarrow \min\left(1.0, \text{stability}(m) + 0.1 \times \min\left(2.0, \frac{\Delta t(m)}{7}\right)\right) \quad (6)$$

The stability update implements a spaced repetition effect: memories accessed after a longer gap receive a larger stability increment (up to a cap at 14 days), while memories accessed in rapid succession gain less stability per access [Cepeda et al., 2006]. Stability is clamped to $[0, 1]$, so a memory that has been frequently accessed over a long period saturates at $S(m) = 1.0$, representing maximum resistance to decay. Figure 1 illustrates the divergence: two memories with identical starting conditions, both retrieved every 7 days, develop markedly different stability and retention profiles depending on whether retrieval is direct or associative. Critically, only the directly retrieved memory accumulates enough stability to cross the core promotion threshold, gaining elevated floor protection that the associatively retrieved memory never earns.

Associative retrieval. When a memory is not a direct match but is *associated* with a directly retrieved memory (see Section 3.7), it receives a smaller boost:

$$\text{accessCount}(m) \leftarrow \text{accessCount}(m) + 1 \quad (7)$$

$$\text{stability}(m) \leftarrow \min\left(1.0, \text{stability}(m) + 0.03 \times \min\left(2.0, \frac{\Delta t(m)}{7}\right)\right) \quad (8)$$

This two-tier structure captures the observation that being directly recalled strengthens a memory more than being passively activated through association, while still acknowledging that any activation at all has a reinforcing effect.

To test whether this divergence is robust to realistic variation in access patterns, we conducted a Monte Carlo analysis with 500 simulation runs. In each run, both memories received the same randomised retrieval schedule: approximately 12 retrieval events (Poisson-distributed) placed at random days within the 90-day window, with new sessions every 14 days. All other parameters were identical to the deterministic case.

The results (Figure 2) confirm that the divergence is structural rather than an artefact of regular scheduling. Directly retrieved memories achieved core promotion in 76.8% of runs (384/500), with a median promotion day of 68 (IQR: 60–76). Associatively retrieved memories achieved core promotion in 0% of runs. At day 90, directly retrieved memories had mean stability 0.985 ± 0.052 , while associatively retrieved memories reached only 0.417 ± 0.044 . The confidence bands in Figure 2 show that even the 10th-percentile direct retrieval trajectory exceeds the 90th-percentile associative trajectory by approximately day 40. This separation is a consequence of the $3.3\times$ boost ratio (0.1 vs. 0.03) compounding over repeated access events; it is not sensitive to the specific scheduling of those events.

3.7 Associative Memory Graph

Memories form bidirectional associations when they are retrieved together or created in the same context. Each association has a weight $w \in [0, 1]$ that strengthens with co-retrieval:

$$w(a, b) \leftarrow \min(1.0, w(a, b) + 0.1) \quad (9)$$

When memory a is directly retrieved, the system checks its associations and returns any associated memory b where $w(a, b)$ exceeds a threshold (default: 0.3), provided b 's own retention score is above a minimum. Associated memories are marked as such in the results and receive the smaller associative boost described above.

This mechanism models spreading activation [Collins and Loftus, 1975]: retrieving a concept activates related concepts, with activation strength proportional to association weight. Over time, frequently co-retrieved memories develop strong bidirectional links, while associations between rarely co-accessed memories weaken. Specifically, association weights decay exponentially based on time since last co-retrieval:

$$w(a, b) \leftarrow w(a, b) \times \exp\left(\frac{-\Delta t_w}{90}\right) \quad (10)$$

where Δt_w is days since the association was last strengthened (i.e., since a and b were last co-retrieved). The 90-day time constant means unused associations fade slowly but steadily: after 90 days without co-retrieval, an association retains approximately 37% of its weight; after 180 days, approximately 13%. Associations that fall below the retrieval threshold (0.3) no longer trigger associative retrieval from one endpoint alone. They can strengthen again if both endpoint memories are later retrieved together through direct semantic search, same-context ingestion, deep recall, or another hot-memory path.

3.8 Consolidation

The system includes a consolidation process that clusters fading memories (retention below 0.2) by semantic similarity and compresses groups of five or more into summary memories. However, unlike typical consolidation approaches that discard originals immediately after compression, our revised policy preserves them through the tiered storage mechanism described below. Original memories are clamped to the regular floor (0.02), marked with metadata linking them to their superseding summary (`supersededBy=<summaryId>`), and migrated to cold storage. In normal retrieval, superseded originals are not eligible as direct hits. They surface only as associated children when their superseding summary is retrieved, providing a reversible path back to the raw details during the cold-storage window.

3.9 Tiered Storage

A fundamental tension exists in any memory system that avoids pruning by decay alone: the stored corpus grows monotonically. While individual text records are cheap to store, vector similarity search degrades as the candidate pool expands. We address this with a two-tier architecture inspired by the neuroscientific distinction between memory *availability* and *accessibility* [Tulving and Pearlstone, 1966].

Tulving and Pearlstone’s foundational finding demonstrated that memories can be intact in storage (available) yet fail to be retrieved (inaccessible) without appropriate cues. Recent neuroscience has reinforced this view: engrams corresponding to apparently forgotten memories persist in “silent” states that resist natural reactivation but can be recovered through artificial stimulation [Ryan and Frankland, 2022, Guskjolen et al., 2024]. At the systems level, the Complementary Learning Systems framework [McClelland et al., 1995] posits a fast-learning, fast-decaying hippocampal store that gradually transfers memories to a slower, more stable neocortical store—a two-tier architecture with migration driven by time and consolidation.

Our tiered storage implements a computational analogue. The **hot store** is a vector-indexed table supporting embedding similarity search: this is the agent’s working memory, containing all active, recently accessed, and core memories. The **cold store** is a plain key-value table in the same database—no embeddings indexed, no similarity search, accessed only by ID. Memories migrate from hot to cold when their retention has remained at floor for a configurable threshold (default: 7 consecutive days with zero retrievals). Core memories are exempt from migration. Superseded originals from consolidation are migrated to cold storage immediately.

The cold store is not a graveyard. When a hot memory is retrieved and has associations pointing to cold IDs, the system performs an $O(1)$ lookup by identifier and applies associative

scoring. If a cold memory surfaces through this path and receives a retrieval boost, it is migrated back to the hot store—re-indexed for similarity search and fully reactivated. This bidirectional migration mirrors the concept of silent engrams: memories that are available but not accessible through normal retrieval, yet recoverable when the right associative cue provides a path back. Deployments can additionally configure a long cold-storage TTL (default: 180 days), after which a cold record is converted into a lightweight archival stub; stubs remain excluded even in deep recall. The benchmark results reported below operate within horizons shorter than this TTL.

Deep recall mode. In neuroscience, “silent engrams” are memory traces that persist biologically but cannot be reactivated through natural retrieval cues; they require artificial stimulation (e.g., optogenetic activation of dentate gyrus neurons) to produce memory expression [Ryan and Frankland, 2022, Guskjolen et al., 2024]. Our superseded originals in cold storage are computationally analogous: they exist, they are available in Tulving’s sense, but they are inaccessible through normal similarity search because they have been removed from the vector index.

Deep recall mode provides the artificial stimulation. Normal search excludes cold memories, superseded originals, expired transient records, and lightweight stubs. The system exposes an optional retrieval parameter (`deep_recall: true`) that temporarily includes cold memories and superseded originals in the direct candidate set, while stubs remain excluded. Superseded records are memories replaced by a newer fact or a consolidation summary; expired transient records are time-scoped plans or temporary states whose validity window has ended; stubs are archival placeholders with content and pointers but no searchable embedding. Superseded and expired records receive a multiplicative penalty (default: $0.5\times$ their normal score) to ensure they surface only when the query strongly matches. This is implemented as a parameter on the canonical `search()` function rather than a system-wide mode, keeping the API stateless and allowing the calling agent (or user) to decide when exhaustive recall is warranted—for example, when the user asks “what exactly did I say about X three months ago?” and a consolidated summary is insufficient. The compatibility `retrieve()` API delegates to `search()` and maps results back to the older scored-memory shape.

This design reflects the broader principle from Tulving and Pearlstone’s work that apparent forgetting often represents a retrieval problem, not a storage problem [Tulving and Pearlstone, 1966]. By changing the retrieval conditions (expanding the candidate set), information that was functionally forgotten becomes accessible again—without requiring that it pollute normal retrieval at all other times.

Scaling behaviour. To evaluate whether tiered storage bounds the hot index across realistic usage patterns, we simulated one year of operation under six scenarios varying the memory ingestion rate (2–10 memories/day), query frequency (1–12 queries/day), and results per query (3–4). Figure 3 shows the results. Across all scenarios, the hot index stabilises at 7–11% of total memories by day 365. In the moderate case (4 memories/day, 3 queries/day), the hot index settles at 124 entries (7.0% of 1,774 total). Even in the most challenging scenario—bursty ingestion at 10 memories/day with only 2 queries/day—the hot index reaches 394 (8.7% of 4,529 total) and plateaus. The consistency of this range across diverse access patterns suggests that the bounded hot index is a structural property of the architecture rather than an artefact of any particular usage profile.

Caveats. The preservation-first assumption underlying this architecture is debatable; we discuss the counterevidence from intrinsic forgetting research in Section 8.

4 Design Comparison with FadeMem

FadeMem [Wei et al., 2026] is the most closely related system, sharing our Ebbinghaus-inspired foundation. Table 3 summarises the key differences.

Design Choice	FadeMem	cognitive-memory (ours)
Decay endpoint	Memories can decay to removal; hard capacity caps (1000 LTM, 500 STM)	Decay never sets retention score to zero; floors at 2% (regular) and 60% (core), with full records eventually degrading to lightweight stubs after cold-storage TTL
Memory hierarchy	Dual-layer (STM/LTM) with promotion criteria	Flat store with category-based decay rates; core as a category, not a layer
Core memory	Not implemented as an explicit category; high-importance memories receive elevated retention through scoring but without guaranteed floor protection	Dual-path: LLM tagging at extraction + emergent promotion through use
Retrieval reinforcement	Access strengthens retention	Two-tier: direct retrieval (+full boost) vs. associative drag-along (+partial boost)
Associative linking	Not described as a separate mechanism; memory fusion serves a related consolidation role	Bidirectional weighted associations with co-retrieval strengthening
Index scaling	Hard capacity caps (1000 LTM, 500 STM); excess memories pruned	Tiered hot/cold/stub storage; hot index bounded at $\sim 10\%$ of total in simulation (Figure 3)
Conflict resolution	LLM-guided classification into contradiction, update, overlap	LLM-guided detection adopting FadeMem’s taxonomy; contradicted core memories are demoted to regular floor (Section 8)
Evaluation	Benchmark-based (MSC, LoCoMo, LTI-Bench)	Production deployment plus LoCoMo, LongMemEval-S, and a controlled LTI-Bench (Section 6)

Table 3: Design comparison between FadeMem and **cognitive-memory**.

The most significant philosophical difference is about deletion timing. FadeMem treats memory removal as a feature: by pruning irrelevant memories, it reduces storage by 45% and improves retrieval precision. Our position is that the storage cost of retaining faint text memories for a configurable dormancy window is often modest, and the unpredictability of future relevance makes early deletion a risky optimisation. We optimise retrieval precision not by immediately removing faint memories but by down-weighting them through the multiplicative scoring in Equation 4, migrating them out of the hot index, and eventually degrading long-dormant cold records to lightweight stubs.

FadeMem’s strengths relative to our system include its LLM-guided conflict resolution—which our system now adopts in deferred form (Section 8)—and its memory fusion mechanism for consolidating related information.

5 Implementation

The architecture is implemented as `cognitive-memory`, an open-source TypeScript package with a Python SDK wrapper. The system is adapter-based: it defines interfaces for vector storage and embedding providers, with default implementations for common backends.

5.1 Adapter Pattern

```
interface MemoryStore {
  store(memory: Memory): Promise<void>
  query(embedding: number[], limit: number): Promise<Memory[]>
  update(id: string, fields: Partial<Memory>): Promise<void>
}

interface EmbeddingProvider {
  embed(text: string): Promise<number[]>
}
```

This allows the core decay, boosting, and association logic to remain independent of the storage backend. The system has been tested with Pinecone, pgvector, and in-memory stores.

5.2 Production Deployment

The system is deployed in `blah.chat`, a consumer chat application where users interact with LLM-based agents across multiple sessions. In this deployment, the memory system:

- Extracts memories from each conversation turn using an LLM call
- Stores them with initial importance scores and category assignments
- Retrieves relevant memories at the start of each agent turn, injecting them into the system prompt
- Runs decay recalculation periodically (currently on each retrieval event)
- Checks for emergent core memory promotion after each retrieval cycle

The system is currently serving real users and includes an append-only audit log that records every state change: retrieval boosts, decay recalculations, association updates, core memory promotions, and consolidation events. Each log entry captures the memory ID, event type, old and new values, and a timestamp. This instrumentation enables post-hoc analysis of individual memory lifecycles and is the basis for the production-data study planned in Section 9; the benchmark evaluation in Section 6 uses the same SDK code path.

6 Evaluation

We evaluate `cognitive-memory` on three benchmarks: LoCoMo [Maharana et al., 2024] for multi-session conversational QA, LongMemEval-S [Wu et al., 2025] for long-horizon memory tasks across six question categories, and a controlled synthetic long-term-interaction harness (LTI-Bench) targeted at the architectural claims of this paper. We additionally report ablations on individual feature contributions, an oracle evidence-context condition, and a judge-reliability check.

6.1 Setup

Models. Memory extraction and answer generation use `gpt-4o-mini`; embeddings use `text-embedding-3-small`. LongMemEval-S and LTI-Bench both use `gpt-4o-2024-08-06` as the judge, matching LongMemEval’s official setup and keeping judge-mediated results comparable across benchmarks.

SDK and provenance. Benchmark provenance spans two SDK states. LongMemEval-S, LTI-Bench, oracle, retrieval-quality, efficiency, judge-reliability, and ablation results come from the May 2026 current-refresh artifacts, where the logs record editable local SDK source observed as package version 0.3.0, commit 905aba7, and a dirty worktree after the v6 behavior fixes. The LoCoMo headline result is updated from the Phase 5 full-LoCoMo head-to-head using the v0.5 tuned SDK defaults shipped in `cognitive-memory-sdk` commit 707758d, with benchmark artifacts at commit 82f08c2. The current public package release is 0.5.0; reported numbers should be traced to the logged artifacts, not inferred from package version alone. The canonical API is `search()`; `retrieve()` is a compatibility wrapper. Normal search excludes cold, superseded, and stub records. Deep recall includes cold and superseded records, while still excluding stubs. Retrieval scoring uses $\text{sim}(m, q)R(m)^\alpha$ with default $\alpha = 0.3$. The benchmark repository records exact commands, configurations, timestamps, output paths, and SDK/worktree provenance in `experimentlog_v2.md`, `docs/milestones/`, `tuning/runs/runs.jsonl`, and per-trial artifacts.

Hardware. All runs are on a single machine (Apple Silicon, 128 GB RAM). Throughput is bounded by OpenAI API latency rather than local compute.

Adapter configuration. The default pipeline uses exponential decay (with a separate condition for power-law), graph expansion at one hop, rerank enabled with factor three, and hybrid lexical+dense search disabled (we ablate this). LoCoMo uses top- k of 60 with dual-perspective ingestion and the Mem0-style answer prompt. LongMemEval-S uses top- k of 20 with deep recall and rerank. The Phase 5 LoCoMo comparison isolates the v0.5 default changes that matter under the benchmark adapter: `associative_boost` from 0.03 to 0.05 and semantic base decay from 120 to 240 days. The SDK’s `core_session_threshold` also moved from 3 to 2, but the benchmark adapter already pinned it to 2 in both arms.

6.2 LoCoMo

LoCoMo is a multi-session conversational QA benchmark with 10 conversations averaging 300 turns and 9k tokens, evaluated on five question categories: single-hop, multi-hop, temporal, open-domain, and adversarial. We score using token F1 against gold answers, following the benchmark’s protocol.

Table 4 reports our headline result against published baselines and the paper-faithful v0.4 configuration. The v0.5 tuned defaults improve full-LoCoMo F1 from 44.4% to 46.2% (+1.87pp) and LLM-judge accuracy from 58.6% to 61.3% (+2.73pp) under the same harness configuration. Multi-hop F1 reaches 51.3%, approximately $1.8\times$ Mem0’s reported multi-hop F1 of 28.4% on the same benchmark. The multi-hop category is the hardest in LoCoMo, requiring the system to combine evidence from multiple sessions; this is also the category where decay-floor and emergent-core mechanisms have the largest plausible benefit, since stale-but-relevant evidence from earlier sessions must remain retrievable. The tuning gain is not uniform by category: temporal F1 falls from 24.1% to 22.3%, while multi-hop and open-domain gains drive the overall improvement.

System	Single	Multi	Temporal	Open	Overall
Mem0 [Chhikara et al., 2025]	—	28.4	—	—	—
cognitive-memory v0.4 (paper-faithful)	35.9	46.9	24.1	48.5	44.4
cognitive-memory v0.5 (tuned)	36.1	51.3	22.3	50.4	46.2

Table 4: LoCoMo F1 (%) on the standard category 1–4 subset ($n = 1540$). The v0.4 and v0.5 rows are from the Phase 5 full-LoCoMo head-to-head artifacts in `tuning/runs/phase5/`. Mem0’s published multi-hop value is the directly comparable literature number available from Chhikara et al. [2025]; other cross-paper LoCoMo comparisons use different settings and are treated as positioning context rather than controlled baselines.

6.3 LongMemEval-S

LongMemEval-S is the Small variant of LongMemEval, with 500 questions across six task types and approximately 53 haystack sessions per question. Following the benchmark’s protocol, we use `gpt-4o-2024-08-06` as the judge and report task-averaged accuracy. LongMemEval comparisons are sensitive to backbone model, judge model, retrieval budget, and whether the system uses a single-stage memory layer or a multi-stage reasoning architecture; we therefore treat cross-paper comparisons as positioning context rather than strict leaderboard evidence.

Table 5 reports per-task and overall accuracy. We achieve 71.6% task-averaged accuracy. The closest concurrent single-stage baseline at the time of running, ENGRAM [Patel and Patel, 2025], reports 71.4%; our result is within 0.2pp of it without any benchmark-specific tuning. Other recent or differently configured systems—including TiMem [Li et al., 2026], EverMemOS [Hu et al., 2026], MAGMA [Jiang et al., 2026], Hindsight [Latimer et al., 2025], and All-Mem [Lv et al., 2026]—now exceed or complicate both ENGRAM and our result. We discuss this positioning further in Section 8.

Task	n	Accuracy (%)
single-session-user	70	85.7
single-session-assistant	56	76.8
single-session-preference	30	46.7
multi-session	133	69.9
temporal-reasoning	133	64.7
knowledge-update	78	85.9
Task-averaged	—	71.6
Overall	500	72.6
Abstention accuracy	—	90.0
Full-context baseline (published)	—	56.2
ENGRAM [Patel and Patel, 2025] (concurrent)	—	71.4
TiMem [Li et al., 2026] (recent comparison)	—	76.88
EverMemOS [Hu et al., 2026] (recent comparison)	—	83.0

Table 5: LongMemEval-S per-task accuracy and task-averaged accuracy. Our task-averaged result is competitive with concurrent ENGRAM-class single-stage memory systems and outperforms the published full-context baseline by 15.4pp; other systems and different backbone/settings exceed this. Hindsight reports substantially higher LongMemEval-family results under different settings, but is omitted from the table to avoid treating that number as a controlled LongMemEval-S baseline. The single-session-preference category is our weakest, suggesting that short-window preference extraction is a capability gap rather than a long-horizon retention issue.

6.4 Oracle Evidence Context

To separate retrieval quality from answer-generation quality, we ran an oracle evidence-context condition on LoCoMo: for each question, the ground-truth evidence sessions are provided directly as context, bypassing the memory system. Using the same Mem0-style answer prompt and `gpt-4o-mini` answer model, this condition achieves F1 of 63.9% (LoCoMo scoring) / 61.1% (Mem0 scoring). This is not a true upper bound on the task, since the answer model can still fail to use perfect evidence. It is a same-model, same-prompt evidence condition that contextualizes the tuned memory-system F1 of 46.2%, with the remaining gap attributable to a combination of retrieval quality (Section 6.7) and answer-generation noise.

6.5 Decay Comparison: Exponential vs Power-Law

Power-law forgetting curves often fit human memory data better than exponential at long timescales [Murre and Dros, 2015, Wixted, 2004]. We added a power-law option to the decay model and compared the two on LoCoMo conv 0 with otherwise default v6 configuration.

Power-law decay achieves F1 of 29.5% on conv 0 versus exponential’s 25.0%, a +4.6pp improvement in this controlled decay comparison. This run isolates the decay functional form under a narrower configuration than the ablation table below. It is therefore a separate sensitivity check, not a duplicate of Table 6. We retain exponential as the default in the published SDK because the gain is single-conversation and we have not characterised it across the full corpus.

6.6 Ablations

Table 6 reports preliminary per-feature ablations on LoCoMo conv 0. Each row toggles one feature relative to the v6 default configuration; F1 is on the same conversation across conditions. This is a broader default-pipeline ablation than the isolated decay comparison in Section 6.5, which is why the power-law absolute scores differ. In this single-conversation slice, power-law decay is the largest positive contributor, followed by rerank and hybrid search; graph expansion is effectively neutral. These results motivate further full-corpus ablations rather than establishing causal effects across LoCoMo as a whole.

Feature	Off (F1 %)	On (F1 %)	Δ (pp)
Power-law decay (vs exponential)	42.9	46.1	+3.2
Rerank ($\times 3$)	43.8	45.7	+1.9
Hybrid search (BM25 + dense)	42.3	43.9	+1.7
Graph expansion (1 hop)	43.1	43.2	+0.0

Table 6: Preliminary per-feature ablations on LoCoMo conv 0 under the default pipeline. Power-law decay is the largest single contributor in this slice; reranking and hybrid search are also positive, while graph expansion is approximately neutral. These are point estimates on a single conversation; full-corpus multi-seed ablations are future work (Section 9).

6.7 Retrieval Quality

LoCoMo annotates each question with the ground-truth dialog evidence (e.g., session-turn IDs). We post-process the per-question retrieved memory contents and compute Recall@ k : the fraction of questions where any retrieved memory at top- k contains the evidence text (Table 7). Coverage is 99.7% (1535 of 1540 questions have annotated evidence).

The discrepancy between Recall@60 (35.6%) and our tuned overall F1 (46.2%) is informative but not conclusive. The lexical evidence-containment metric likely underestimates semantically

k	Recall@ k (%)
5	24.8
10	28.5
20	31.7
60	35.6

Table 7: Evidence Recall@ k on LoCoMo ($n = 1535$ questions with evidence annotations). Even at $k = 60$, only 35.6% of questions have any retrieved memory containing the gold evidence under lexical containment matching. This likely undercounts semantically useful paraphrases and consolidated evidence, but still indicates retrieval is a bottleneck.

useful retrieval, especially when memories are paraphrased or consolidated. At the same time, the low recall indicates that retrieval remains a major bottleneck. The oracle evidence-context condition in Section 6.4 also leaves answer-generation and answer-utilisation headroom, so the current evidence does not isolate retrieval as the sole limiting factor.

6.8 Efficiency

Table 8 reports per-stage timing from the SDK trace, aggregated across the LoCoMo run. Vector search is sub-100ms; scoring is sub-millisecond. Extraction (one LLM call per ingested turn) dominates wall time at ~ 16 s per turn. Rerank and answer generation timing live outside the SDK trace—rerank is in the adapter layer, answer is in the eval harness—and are deployment-dependent.

Stage	Mean (ms)	p50 (ms)	p95 (ms)
Extraction (per turn)	15,611	14,750	24,500
Embedding (per turn)	375	300	800
Vector search (per query)	54	55	67
Scoring (per query)	0.61	0.58	0.71

Table 8: Per-stage timing on LoCoMo. Extraction is the dominant cost; vector search and scoring are negligible. Rerank and answer generation are not included (deployment-dependent, not in the SDK trace).

6.9 Judge Reliability

The LoCoMo headline F1 numbers depend on a token-F1 score against gold answers, which is robust on its own. LongMemEval-S and LTI-Bench are more load-bearing because they are judge-mediated. As a local sanity check, we sampled 50 question-answer pairs stratified by category and correctness (8 buckets) from the LoCoMo run and re-judged with an alternative “EQUIVALENT vs DIFFERENT” prompt formulation.

We observe Cohen’s κ of 0.879 with 94% raw agreement; 3 of 50 pairs disagreed. This supports local prompt stability for the sampled checks, but it is not a full validation of the LongMemEval-S judge distribution and does not remove broader cross-paper comparability concerns around judge model, rubric, answer format, and retrieval budget.

6.10 LTI-Bench: Architectural Test

The benchmarks above measure end-to-end accuracy on real conversational data. LTI-Bench is a controlled synthetic harness designed instead to exercise the architectural claims of this paper: decay floors, emergent core promotion, conflict-driven supersession, revival of faint memories, and time-aware retrieval.

Scenario. A 30-day single-user interaction with 28 facts ingested across 28 daily sessions. Facts are stratified by importance:

- 8 *critical* facts (identity, medical, family, residence) accessed multiple times across the window;
- 8 *contextual* facts (project, hobby, recurring appointments) with light access patterns;
- 8 *trivial* facts mentioned once and never re-accessed (testing decay floors);
- 4 *updated* facts that get superseded mid-window (testing conflict resolution).

The harness probes the system with 42 questions across eight probe types. Sessions through day D are ingested before any probe at day D fires (*time-stepped* ingestion), so probes for facts that get superseded later in the window can correctly query the original fact at the right time.

Scoring. Each probe is scored both by token F1 against a reference answer and by an LLM judge (`gpt-4o-2024-08-06`, the same model and CORRECT/WRONG prompt formulation as LongMemEval-S). We treat the judge as the headline metric and F1 as supplementary; substring-style scoring is unsuitable for a controlled benchmark with paraphrased answers (a memory probe that returns “April 1, 2024” against a reference of “April 1st” is substantively correct).

Results. Table 9 reports per-category accuracy and F1. Overall accuracy is 88.1% across 42 probes; F1 is 69.7%.

Probe Category	n	Accuracy (%)	F1 (%)
core_persistence	8	100.0	93.3
decay_trivial	6	100.0	64.2
contextual_retention	6	100.0	84.5
temporal_before_update	4	100.0	85.0
temporal_after_update	4	100.0	67.7
conflict	4	75.0	67.7
revival	5	60.0	42.9
associative	5	60.0	38.7
Overall	42	88.1	69.7

Table 9: LTI-Bench per-category results ($n = 42$ probes total, single seed, SDK v0.3.0). The architecture passes core persistence, contextual retention, decay floors (probed via direct queries on never-re-accessed facts), and time-aware retrieval at 100%. Conflict remains mostly correct; revival and associative retrieval are the weakest categories at 60%. Cross-factor “what do you know about X ” queries often return a subset of the relevant cluster rather than the full cluster.

Architectural reading of the results.

- *Decay floors hold.* 100% of trivial facts mentioned once and never re-accessed are recoverable when probed directly at day 30, supporting the floor-retention claim.
- *Critical retention is total.* 100% of identity-critical facts are recovered, against a comparison point of 82.1% reported by FadeMem on a similar synthetic decay scenario.
- *Emergent core promotion fires aggressively.* 66 of 85 stored memories (78%) reach core status by day 30, including all 8 explicitly-tagged critical facts. The 78% rate is high enough to flag in Section 8 as a possible threshold-tuning issue.

- *Conflict-driven supersession works.* All four updated facts return the post-update version at probe time after day 30; all four return the original (pre-update) version when probed at the time it was current (day 10–15 probes).
- *Revival is partial.* 3 of 5 revival probes correctly retrieve a memory mentioned once and never re-accessed when prompted with an oblique cue. The failures show that decay floors preserve recoverability under direct probes, but oblique cues still depend on retrieval phrasing and ranking.
- *Associative retrieval is partial.* The two associative failures reflect a real limitation: a query like “what do you know about my family?” surfaces one of three family-related facts rather than the full cluster. A direct probe for any individual family fact succeeds; the failure is specifically in cross-fact recall, suggesting that the current association mechanism is an architectural scaffold whose downstream retrieval benefit is not yet strongly demonstrated.

The 42-probe sample is too small for strong statistical claims, and the scenario is hand-authored, so LTI-Bench is best read as a confirmatory architectural test rather than a generalisation benchmark. The 78% core-promotion rate should be read as a likely threshold-tuning failure mode, not as a success criterion. We report LTI-Bench because the benchmark categories above (LoCoMo, LongMemEval-S) measure end-to-end QA performance and do not directly probe the lifecycle properties this paper argues for.

7 Responsible Memory and User Control

Agent memory systems store facts about people: names, family relationships, preferences, work context, locations, medical details, and other identity-bearing information. A preservation-first architecture therefore needs product and deployment constraints, not only retrieval mechanics. The SDK treats deletion, stubbing, and deep recall as policy surfaces that applications must expose deliberately.

In production deployments, users should be able to inspect stored memories, delete individual memories, disable memory, and request deletion of cold records and stubs as well as hot records. Sensitive categories such as medical, legal, financial, identity, and location facts should require stricter extraction policy, shorter retention defaults, or explicit user confirmation depending on the application domain. Cold storage and reversible consolidation should not be interpreted as a right to retain everything indefinitely: they are mechanisms for avoiding accidental information loss within a configured dormancy window, not a substitute for consent, data minimisation, encryption, access control, and retention policy.

Deep recall is also a user-control issue. Because it intentionally widens retrieval to include cold and superseded records, applications should reserve it for explicit user requests, audit workflows, or agent actions with clear justification. A consumer-facing agent should not silently use deep recall to surface sensitive or previously superseded memories in ordinary conversation. The benchmark harness exercises deep recall as an architectural capability; deployment policy must decide when that capability is appropriate.

8 Limitations and Open Questions

We are candid about the current limitations of this work:

Most parameter choices remain heuristic. The v0.5 SDK release empirically tunes three defaults: `associative_boost` moves from 0.03 to 0.05, semantic base decay from 120 to 240 days, and `core_session_threshold` from 3 to 2. These changes are grounded in the Phase 0g–5

logs and validated by the full-LoCoMo head-to-head in Table 4. Other values—the direct boost of 0.1, decay floors of 2% and 60%, core promotion stability/access thresholds, the $0.5\times$ deep recall penalty, the 90-day association decay constant, and the consolidation similarity threshold of 0.7—remain informed guesses based on cognitive science analogies and manual tuning. This paper is an architecture description; a subsequent empirical study using production audit logs will examine whether these values are reasonable and where they need adjustment.

Deferred conflict handling. Conflict handling is implemented as a queued maintenance process rather than an inline ingestion step. During ingestion, highly similar cross-session memories are queued as conflict candidates; `tick()` processes the queue with an LLM classifier using FadeMem’s contradiction/update/overlap taxonomy [Wei et al., 2026]. For contradictions or updates, the older record is linked to the newer one via `contradictedBy`; if the older record was core, its category is demoted so its floor drops from 60% to 2%. This avoids the $O(N^2)$ inline LLM bottleneck we observed during early LoCoMo runs, while still allowing corrected facts to supersede stale ones. The limitation is operational rather than conceptual: conflict resolution depends on maintenance cadence and queue processing, so a newly contradicted fact may persist until the next tick.

Single-seed evaluation. The benchmark numbers reported in Section 6 come from a single run per condition; we do not characterise variance across seeds. Concurrent systems we compare against [Chhikara et al., 2025, Wei et al., 2026, Patel and Patel, 2025] also report single-seed results, so the comparison is internally consistent, but absolute differences within a few percentage points should be read accordingly. Multi-seed evaluation is planned future work.

LongMemEval-S only. We evaluate on the Small variant of LongMemEval (500 questions, ~ 53 haystack sessions per question) and not on the Medium or Oracle variants. The Medium variant has roughly an order of magnitude larger haystack and would require multi-day wall-clock time and substantially higher API spend; we treated single-S as sufficient because it placed the system within 0.2pp of the closest concurrent baseline at the time of running (ENGRAM, Patel and Patel 2025). Recent or differently configured systems posted around our publication window—TiMem [Li et al., 2026], EverMemOS [Hu et al., 2026], MAGMA [Jiang et al., 2026], Hindsight [Latimer et al., 2025], and All-Mem [Lv et al., 2026]—now exceed or complicate our LongMemEval-S/LoCoMo positioning; we discuss this in Section 6.

Ablations on a single LoCoMo conversation. The feature ablations in Section 6 (hybrid search, graph expansion, rerank, decay model) are run on conversation 0 only, not the full ten-conversation corpus. This was a cost choice: each ablation condition runs the full memory pipeline and the differences we observe are point estimates, not population estimates. Effect signs and rough magnitudes are likely informative; precise pp values are not.

Controlled LTI-Bench is small-sample and synthetic. Our LTI-Bench harness uses a hand-authored 30-day scenario with 28 facts and 42 probes across 8 probe types (Section 6.10). The scenario gives controlled access to the architectural properties the paper claims (decay floors, emergent core, conflict-driven supersession, time-aware retrieval), but the per-category sample sizes ($n=4-8$) are too small to support strong statistical claims about general-purpose conversational performance. We treat LTI-Bench as a confirmatory architectural test, not a generalisation benchmark.

Run provenance. Active benchmark numbers come from two logged artifact families: the May 2026 current-refresh artifacts for LongMemEval-S, LTI-Bench, oracle, retrieval, efficiency, judge reliability, and ablations; and the Phase 5 full-LoCoMo head-to-head for

the tuned LoCoMo headline. The provenance source is therefore the artifact path plus commit/log chain, not the package version string alone. The final LoCoMo row traces to `tuning/runs/phase5/v05_tuned/`, benchmark commit `82f08c2`, and SDK commit `707758d`; current-refresh results trace to the paths listed in Code Availability.

Consolidation clustering is underspecified. As noted in Section 3.8, the consolidation mechanism clusters fading memories by semantic similarity and compresses groups into summaries. The current implementation computes pairwise cosine similarity between embeddings of fading memories and groups those exceeding a threshold (default: 0.7); the grouping size of five is a configurable parameter chosen to balance summary coherence against information density. However, the LLM that performs compression lacks a principled framework for deciding what to preserve and what to discard. It is making editorial judgments about salience without ground truth, and the resulting summaries are lossy even though the originals are now retained in cold storage.

The preservation-first assumption is debatable. As noted in Section 3.9, our preservation-first policy follows the accessibility hypothesis: forgetting often reflects retrieval failure rather than storage loss [Tulving and Pearlstone, 1966]. There is also evidence for genuine *intrinsic forgetting*, where active molecular mechanisms degrade memory traces [Davis and Zhong, 2017]. Some memories may be erased, not merely made inaccessible. Our policy is pragmatic: text storage is cheap, information loss is irreversible, and the SDK can separate retention decay from storage policy. Decay floors prevent score collapse, cold migration controls index size, and a configurable cold-storage TTL can convert very old cold records into stubs.

Core memory thresholds remain unsettled. The thresholds for emergent core promotion (10 accesses, stability ≥ 0.85 , and a configurable cross-session threshold) are only partially empirically validated. Phase 2 suggests the old threshold of 3 sessions underperforms thresholds of 1–2 in the joint search, and v0.5 ships a threshold of 2. Too low, however, and trivial memories get promoted; too high, and genuinely important memories remain unprotected. LTI-Bench makes this risk concrete: 66 of 85 stored memories (78%) become core by day 30. That is likely too aggressive for a production memory system and should be treated as a threshold-tuning problem, not a desirable outcome.

Scalability of associative graph. The associative linking mechanism could produce a dense graph over time. Association weight decay (Section 3.7) mitigates unbounded growth by allowing unused associations to fall below the retrieval threshold, but we have not yet characterised the scaling behaviour at production volumes or determined whether explicit pruning of weak associations is necessary.

9 Future Work

Our immediate priorities are:

1. **Production-data validation and privacy review.** Using audit logs from `blah.chat` to validate or invalidate parameter choices only after appropriate consent, minimisation, and privacy review: do memories actually follow Ebbinghaus-like curves under real usage; how many retrievals does it take for stability to plateau; what is the revival rate for faint memories.

2. **Multi-seed and full-corpus ablations.** Re-running the ablation conditions in Section 6 across multiple seeds and across the full ten-conversation LoCoMo corpus, to convert the current point-estimate effect sizes into population estimates.
3. **Cross-benchmark validation of tuned defaults.** Phase 5 validates v0.5 defaults on full LoCoMo. LongMemEval-S should be rerun as a v0.4/v0.5 head-to-head before claiming the same defaults generalise across benchmark families.
4. **Strengthening associative retrieval.** The LTI-Bench results (Section 6.10) flag associative retrieval as the weakest probe category: cross-fact queries return a subset of the relevant cluster rather than the full cluster. Candidate fixes include explicit graph-walk expansion at retrieval time, broader top- k for associative-typed queries, and a query classifier that routes “what do you know about X ” style queries to a different retrieval policy.
5. **Cross-model generalisation.** Our evaluation uses gpt-4o-mini as the answer model and gpt-4o-2024-08-06 as the judge throughout. We have not characterised how the system performs with a Claude or Llama answer model, or whether the judge-model choice itself influences relative rankings.
6. **LongMemEval-M and Oracle.** Extending evaluation to the larger LongMemEval variants for a stronger comparison against recent systems such as TiMem, EverMemOS, MAGMA, Hindsight, and All-Mem.

10 Conclusion

Forgetting in agent memory is not unsolved, but it is unsettled. Multiple systems now implement decay mechanisms inspired by cognitive science, and no single approach has emerged as dominant. This paper contributes to that active conversation by describing an architecture with several specific design commitments: that memories should not be pruned simply because they are currently inaccessible, that core status should be earnable through use rather than only assignable at extraction, that retrieval should produce differentiated reinforcement depending on whether access was direct or associative, and that associative links between memories should themselves be dynamic.

These commitments are grounded in specific cognitive science mechanisms, implemented in open-source software, and deployed in a production application. Section 6 provides a first empirical pass: on LoCoMo the tuned v0.5 defaults recover an overall F1 of 46.2% with multi-hop F1 of 51.3% ($1.8\times$ Mem0’s reported multi-hop number), improving over the paper-faithful v0.4 row by +1.87pp F1 and +2.73pp LLM-judge accuracy; on LongMemEval-S we reach 71.6% task-averaged accuracy, competitive with ENGRAM-class single-stage memory systems but behind higher-scoring or differently configured architectures published around the same window; preliminary single-conversation ablations show decay-function sensitivity in that slice (+3.2pp on conv 0 in the ablation runner, +4.6pp in the controlled decay comparison); a controlled long-term-interaction harness shows 100% retention of identity-critical facts across a 30-day mixed-access window, while also exposing likely over-promotion to core status. The numbers do not unambiguously settle whether decay floors and emergent core promotion improve downstream task performance over alternative architectures; they do indicate that the architecture is competitive at single-stage memory retrieval and that the specific architectural commitments behave as designed under controlled tests.

Code Availability

The `cognitive-memory` package and the simulation code used to generate Figures 1, 2, and 3 are available at:

- SDK (TypeScript and Python): <https://github.com/planetariescape/cognitive-memory>
- Benchmarks code and per-run JSON results:
<https://github.com/planetariescape/cognitive-memory-benchmarks>
- Paper source and PDF:
<https://github.com/planetariescape/cognitive-memory-benchmarks/tree/main/paper>
- Main LoCoMo artifact: `tuning/runs/phase5/v05_tuned/aggregate.json` and per-conversation `conv*.json`
- LoCoMo v0.4/v0.5 comparison: `tuning/runs/phase5/summary.json`
- Main LongMemEval-S artifact:
`longmemeval/results/current_sdk_20260505/primary.json`
- Main LTI-Bench artifact: `lti/results/current_sdk_20260505/run_1_v2.json`
- Experiment provenance: `experimentlog_v2.md`, `docs/milestones/`, `tuning/runs/runs.jsonl`, and per-trial `tuning/runs/lti-*`
- npm package: `cognitive-memory` (latest public release 0.5.0)
- PyPI package: `cognitive-memory` (latest public release 0.5.0)

The evaluation in Section 6 uses the May 2026 current-refresh artifacts plus the Phase 5 full-LoCoMo v0.4/v0.5 head-to-head recorded in the benchmarks repository. LongMemEval-S and LTI-Bench numbers remain current-refresh results; the LoCoMo headline row is the later v0.5 tuned-default result.

References

- Robert A. Bjork and Elizabeth L. Bjork. A new theory of disuse and an old theory of stimulus fluctuation. *From Learning Processes to Cognitive Processes: Essays in Honor of William K. Estes*, 2:35–67, 1992.
- Nicholas J. Cepeda, Harold Pashler, Edward Vul, John T. Wixted, and Doug Rohrer. Distributed practice in verbal recall tasks: A review and quantitative synthesis. *Psychological Bulletin*, 132(3):354–380, 2006.
- Xuechen Chen et al. Self-evolving agents with reflective and memory-augmented abilities. *arXiv preprint arXiv:2409.00872*, 2024.
- Prateek Chhikara et al. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.
- Allan M. Collins and Elizabeth F. Loftus. A spreading-activation theory of semantic processing. *Psychological Review*, 82(6):407–428, 1975.
- Ronald L. Davis and Yi Zhong. The biology of forgetting—a perspective. *Neuron*, 95(3):490–503, 2017.

- Hermann Ebbinghaus. *Memory: A Contribution to Experimental Psychology*. Teachers College, Columbia University, 1885. Translated by H. A. Ruger and C. E. Bussenius, 1913.
- Jason R. Finley. Expanded taxonomies of human memory. *Frontiers in Cognition*, 2025.
- Uwe Frey and Richard G. M. Morris. Synaptic tagging and long-term potentiation. *Nature*, 385:533–536, 1997.
- Axel Guskjolen et al. Natural forgetting reversibly modulates engram expression. *eLife*, 13: e92860, 2024.
- Chuanrui Hu, Xingze Gao, Zuyi Zhou, Dannong Xu, Yi Bai, Xintong Li, Hui Zhang, Tong Li, Chong Zhang, Lidong Bing, and Yafeng Deng. EverMemOS: A self-organizing memory operating system for structured long-horizon reasoning. *arXiv preprint arXiv:2601.02163*, 2026. <https://arxiv.org/abs/2601.02163>.
- Dongming Jiang, Yi Li, Guanpeng Li, and Bingzhe Li. MAGMA: A multi-graph based agentic memory architecture for AI agents. *arXiv preprint arXiv:2601.03236*, 2026. <https://arxiv.org/abs/2601.03236>.
- Jiazheng Kang, Mingyu Ji, Zhe Zhao, and Ting Bai. Memory OS of AI agent. *arXiv preprint arXiv:2506.06326*, 2025.
- Chris Latimer, Nicolás Boschi, Andrew Neeser, Chris Bartholomew, Gaurav Srivastava, Xuan Wang, and Naren Ramakrishnan. Hindsight is 20/20: Building agent memory that retains, recalls, and reflects. *arXiv preprint arXiv:2512.12818*, 2025. <https://arxiv.org/abs/2512.12818>.
- Jaewoo Lee and Bosung Jang. Memoria: Resolving fateful forgetting problem through human-inspired memory architecture. *arXiv preprint arXiv:2310.03052*, 2023.
- Kai Li, Xuanqing Yu, Ziyi Ni, Yi Zeng, Yao Xu, Zheqing Zhang, Xin Li, Jitao Sang, Xiaogang Duan, Xuelei Wang, Chengbao Liu, and Jie Tan. TiMem: Temporal-hierarchical memory consolidation for long-horizon conversational agents. *arXiv preprint arXiv:2601.02845*, 2026. <https://arxiv.org/abs/2601.02845>.
- Shichun Liu et al. Memory in the age of AI agents: A survey. *arXiv preprint arXiv:2512.13564*, 2025.
- Can Lv, Heng Chang, Yuchen Guo, Shengyu Tao, and Shiji Zhou. All-Mem: Agentic lifelong memory via dynamic topology evolution. *arXiv preprint arXiv:2603.19595*, 2026. <https://arxiv.org/abs/2603.19595>.
- Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13851–13870, 2024. <https://arxiv.org/abs/2402.17753>.
- James L. McClelland, Bruce L. McNaughton, and Randall C. O’Reilly. Why there are complementary learning systems in the hippocampus and neocortex: Insights from the successes and failures of connectionist models of learning and memory. *Psychological Review*, 102(3): 419–457, 1995.
- Jaap M. J. Murre and Joeri Dros. Replication and analysis of Ebbinghaus’ forgetting curve. *PLoS ONE*, 10(7):e0120644, 2015.

- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST ’23)*. ACM, 2023.
- Daivik Patel and Shrenik Patel. ENGRAM: Effective, lightweight memory orchestration for conversational agents. *arXiv preprint arXiv:2511.12960*, 2025. <https://arxiv.org/abs/2511.12960>.
- Vencislav Popov, Iva Marevic, Jan Rummel, and Lynne M. Reder. Forgetting is a feature, not a bug: Intentionally forgetting some things is an effortful process that aids memory for target items. *Psychological Science*, 30(9):1303–1315, 2019.
- Blake A. Richards and Paul W. Frankland. The persistence and transience of memory. *Neuron*, 94(6):1071–1084, 2017.
- Tomás J. Ryan and Paul W. Frankland. Forgetting as a form of adaptive engram cell plasticity. *Nature Reviews Neuroscience*, 23:173–186, 2022.
- Theodore R. Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L. Griffiths. Cognitive architectures for language agents. *Transactions on Machine Learning Research*, 2024.
- Endel Tulving and Zena Pearlstone. Availability versus accessibility of information in memory for words. *Journal of Verbal Learning and Verbal Behavior*, 5(4):381–391, 1966.
- Lei Wei, Xu Dong, Xiao Peng, Niantao Xie, and Bin Wang. Fademem: Biologically-inspired forgetting for efficient agent memory. *arXiv preprint arXiv:2601.18642*, 2026.
- John T. Wixted. The psychology and neuroscience of forgetting. *Annual Review of Psychology*, 55:235–269, 2004.
- Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. Long-MemEval: Benchmarking chat assistants on long-term interactive memory. In *International Conference on Learning Representations (ICLR)*, 2025. <https://arxiv.org/abs/2410.10813>.
- Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. Memorybank: Enhancing large language models with long-term memory. *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024.

Two-Tier Retrieval Boosting: Direct vs. Associative

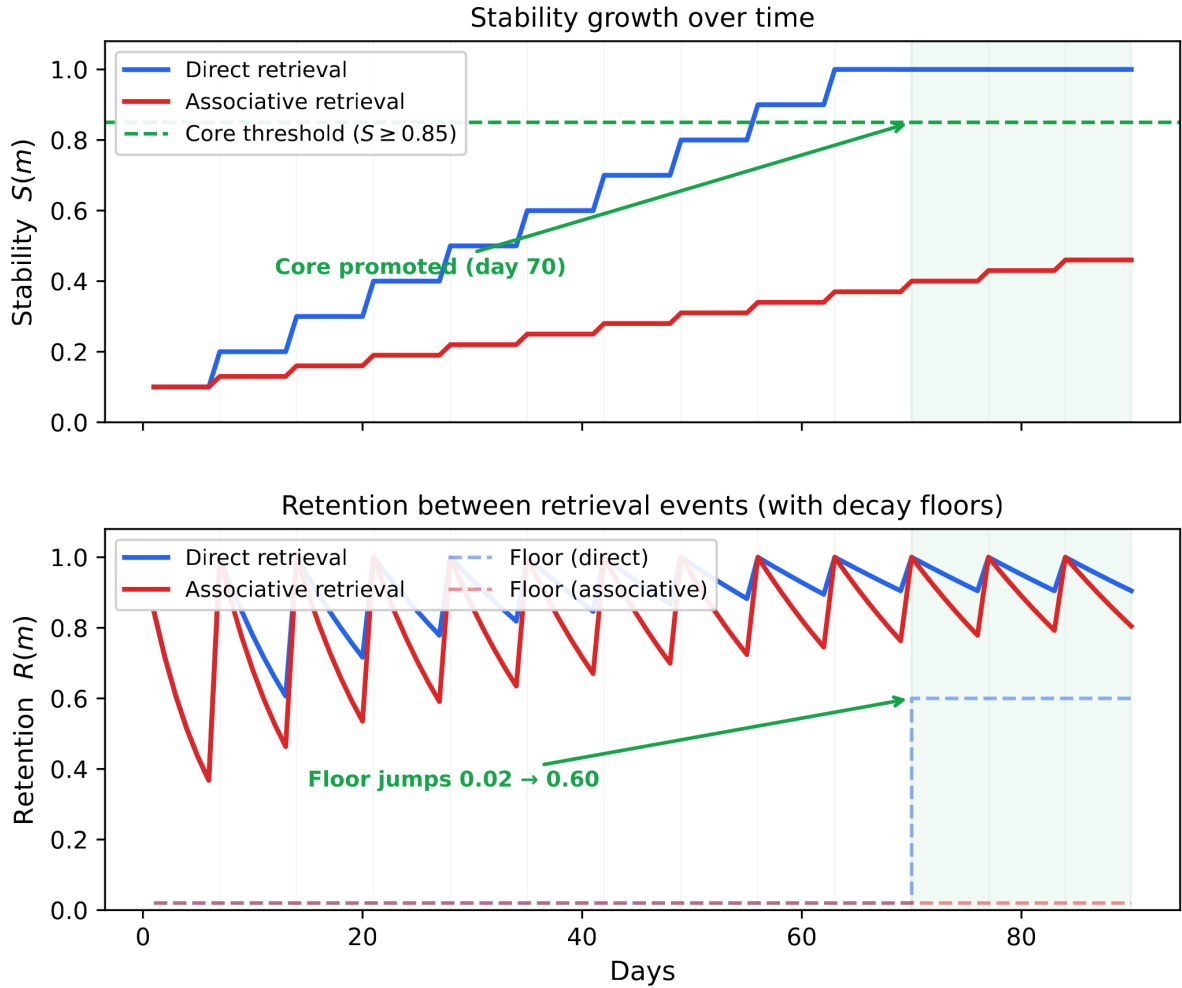


Figure 1: Simulated divergence between direct and associative retrieval over 90 days. Both memories start with identical parameters ($S = 0.1$, importance = 0.5, episodic category) and are retrieved every 7 days, with new sessions every 14 days. **Top:** Stability growth. The directly retrieved memory crosses the core promotion threshold ($S \geq 0.85$) at day 70 (green shaded region), while the associatively retrieved memory reaches only $S = 0.46$ and never qualifies. **Bottom:** Retention between retrieval events. At day 70, the directly retrieved memory's decay floor jumps from 0.02 to 0.60 upon core promotion (blue dashed line), guaranteeing it can never fade below 60% retention while it remains a core memory. The associatively retrieved memory's floor remains at 0.02 throughout. The same 12 retrieval events, differentiated only by boost magnitude, produce fundamentally different long-term outcomes: one memory earns elevated floor protection, the other does not.

Monte Carlo Analysis: Two-Tier Boosting Under Randomised Retrieval

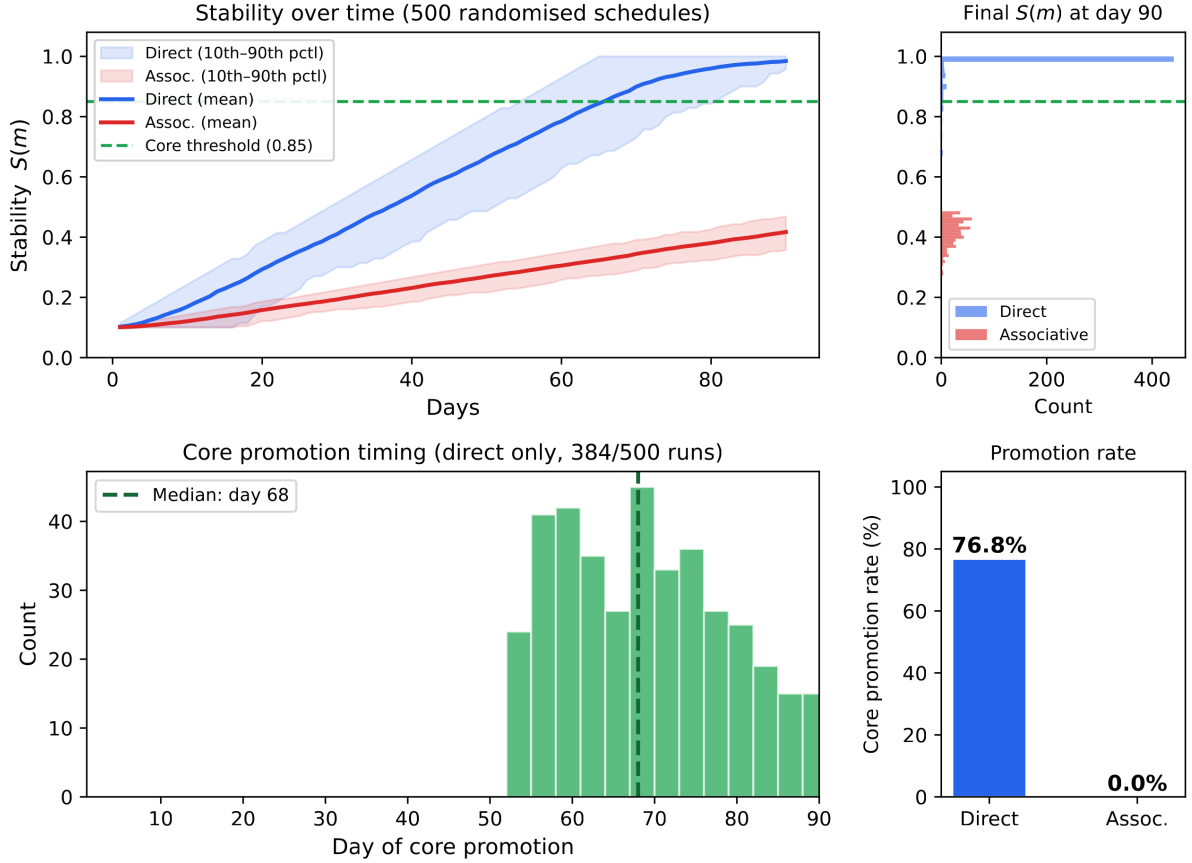


Figure 2: Monte Carlo analysis of two-tier boosting across 500 randomised retrieval schedules (~ 12 Poisson-distributed events per run, sessions every 14 days). **Top left:** Mean stability trajectories with 10th–90th percentile bands. The direct retrieval mean crosses the core threshold around day 60; the associative mean never approaches it. **Top right:** Distribution of final stability at day 90, showing near-complete separation between the two tiers. **Bottom left:** Distribution of core promotion timing for direct retrieval (median: day 68). **Bottom right:** Core promotion rates: 76.8% for direct, 0% for associative.

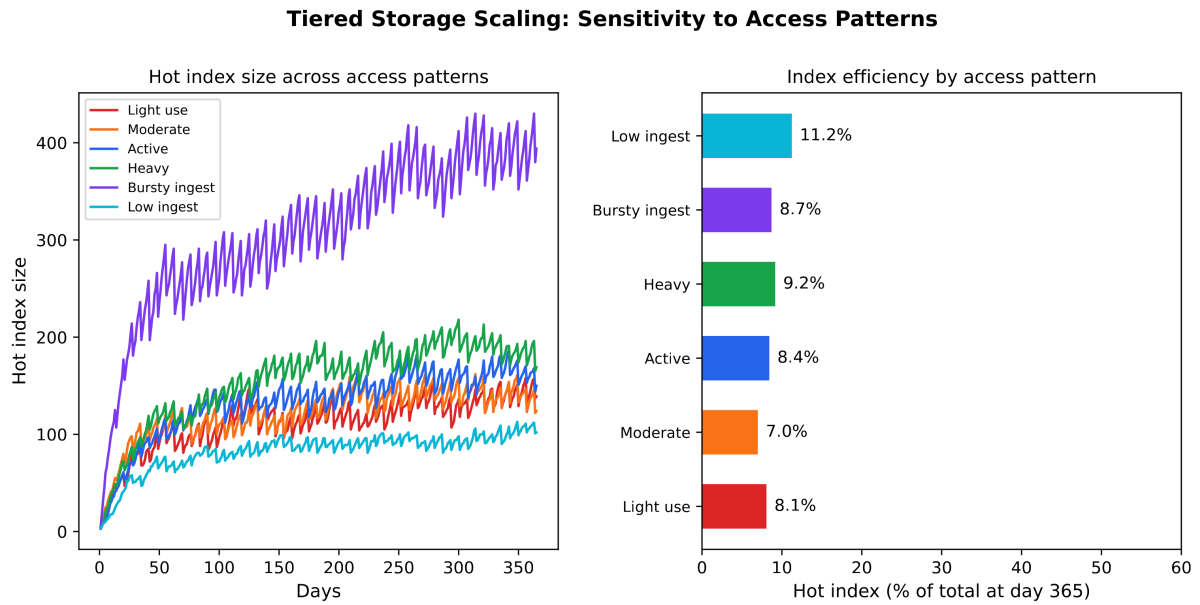


Figure 3: Tiered storage scaling across six access patterns over one simulated year. **Left:** Hot index size over time. All scenarios plateau; absolute size scales with ingestion rate but growth converges to zero. The bursty ingest scenario (10 memories/day, 2 queries/day) shows the most variance but still stabilises. **Right:** Hot index as percentage of total memories at day 365. All scenarios converge to the 7–11% range, indicating that bounded hot index size is robust to access pattern variation.